Report.pdf (content)

ShopSphere Back-End Scalability Audit

Prepared for: Chief Technology Officer, ShopSphere.
Prepared by: Algorithm Engineer (Student Name: E.C. Jithendra Kumar Reddy
Reg. No.: 192472180)
Course reference: CSA06 Design and Analysis of Algorithms, CO1 Assessment Tool 1
Scope: Five back-end modules (M1 to M5), catalogue growth from n = 100,000 to n = 1,000,000 products

1. Executive Summary

ShopSphere's catalogue is expected to grow ten times within a year. I audited five back-end modules to see how each will cope. The main findings are:

- Three modules will fail. The duplicate-listing detector (M1), the recommendation engine in its current design (M4) and the nightly report generator (part of M5) all have cost that grows with the square of the catalogue size. A ten-fold increase in data makes each of them about a hundred times slower.
- The other modules are safe. The API rate limiter (M2), the sorted-index search (S2), the analytics engine (M5) and the proposed M4 redesign all scale linearly or close to it. Each finishes in seconds at one million products.
- The problem is closer than it looks. At today's size the three heavy jobs take about 550 seconds in total, which is already 92% of the 10-minute nightly window. The report generator alone would use up the whole window at about 141,000 products, only about 41% more than today.
- One internal claim was wrong. The team member's statement that M2 is Θ(n log n) is incorrect. The tight bound is Θ(n), and the exact count is 2^⌈log n⌉ − 1.
- The fix is achievable. Replacing the three quadratic modules with Θ(n) or Θ(n log n) designs brings the whole nightly workload at one million products to about 4 seconds in total, down from over 15 hours.

Module	Verdict	Time at 1,000,000 products (10^8 ops/s)	Action
M1 Duplicate detector	Fails window	5,000 s (83 min)	Redesign with hashing
M2 API rate limiter	Safe	0.01 s	Keep
S2 Product search	Safe	20 comparisons per search	Use S2 (not S1)
M4 Recommendation, current	Fails window	20,000 s (5.6 hours)	Adopt the proposed redesign
M4 Recommendation, redesign	Safe	0.21 s	Adopt
M5 Analytics engine	Safe	2.1 s	Keep (optional tuning)
M5 Report generator	Fails window	30,002 s (8.3 hours)	Redesign

Recommended priority: (1) report generator, (2) M4 redesign, (3) M1.

(Insert Output.png here as Figure 1. The left panel shows how each module's cost grows with n against the 10-minute budget, and the right panel shows running time at one million products, with the three failures in red.)

2. Background and Scope

ShopSphere is a growing online shopping platform. The product catalogue holds n = 100,000 products today and is expected to reach n = 1,000,000 within a year. Each night, a set of back-end jobs must finish inside a 10-minute processing window.

I was asked to:

1. understand what each of five modules does and what drives its running time,
2. predict how each behaves as n grows, and
3. recommend which modules must be redesigned.

The five modules are the duplicate-listing detector (M1), the API rate limiter (M2), the product search (M3, with two versions S1 and S2), the recommendation engine (M4) and the sales analytics engine with its nightly report generator (M5).

3. Method and Assumptions

3.1 How each module was analysed

For each module I:

1. identified the input size and the basic operation, which is the step repeated most often and therefore drives the running time,
2. counted how often that operation runs, either exactly (for the loop-based modules) or by solving the recurrence (for the recursive modules),
3. stated the result using asymptotic notation (Θ),
4. converted the operation count into running time using the server speed.

The detailed derivations are in Solution.pdf. This report concentrates on what the results mean for the platform.

3.2 Assumptions

Assumption	Value	Note
Catalogue size today	n = 100,000	Given
Catalogue size in a year	n = 1,000,000	Given
Server speed	10^8 basic operations per second	The exponent is missing in the question paper, so 10^8 is assumed. Results at 10^9 are also shown.
Nightly window	10 minutes = 600 seconds	Given. At 10^8 ops/s this allows about 6×10^{10} operations.
Logarithms	Base 2	

Cost of one basic operation	One unit of time	A simplification; real operations vary in cost.
Jobs run one after another	On one server	Used only for the combined-workload figures.

4. Findings by Module

4.1 M1: Duplicate-Listing Detector

What it does. Every night it compares each product with every later product to find duplicate uploads.

Result. The number of comparisons is exactly $n(n - 1)/2$, which is $\Theta(n^2)$. It does not depend on the data, because every pair is always compared. The extra memory needed is $\Theta(1)$.

n	Comparisons	Time at 10^8 ops/s
100,000	4,999,950,000	50 s
1,000,000	499,999,500,000	5,000 s (83 min)

What it means. The catalogue grows 10 times, but the nightly job grows 100 times (50 s to 5,000 s). The module cannot meet the window at one million products unless the server is about ten times faster, and even then it would use 83% of the window by itself.

Why the design is wasteful. Almost every one of the 5×10^{11} comparisons finds no duplicate. The routine compares products that could never match.

4.2 M2: API Rate Limiter

What it does. It audits request windows of doubling size (1, 2, 4, 8, ...).

Result. The exact number of Audit operations is $2^{\lceil \log n \rceil} - 1$, which lies between $n - 1$ and $2n - 3$, so the cost is $\Theta(n)$. For $n = 1{,}024$ the count is $1 + 2 + 4 + ... + 512 = 1{,}023$, as the formula predicts.

The team member's claim. The claim of $\Theta(n \log n)$ is incorrect. The inner loop runs i times, not n times, and i is small in most passes. The passes form a geometric series dominated by the

last term. The claim would overestimate the work by a factor of about 19 at one million requests (about 2×10^7 against the true 1.05×10^6).

What it means. M2 needs about a million operations at one million requests, roughly 0.01 seconds. It poses no scalability risk and needs no redesign.

4.3 M3: Product Search (S1 and S2)

What it does. S1 scans the unsorted catalogue one entry at a time. S2 searches a sorted index by repeatedly halving the range.

Results.

	S1 (sequential)	S2 (binary)
Best case	1 comparison	1 comparison
Average case	$(n + 1)/2$ if the product exists	about $\log n - 1$
Worst case	n	$\lfloor \log n \rfloor + 1$
Complexity	$\Theta(n)$	$\Theta(\log n)$

If the product exists with probability p, S1 averages $n(1 - p/2) + p/2$ comparisons. For $n = 1{,}000{,}000$ and $p = 0.5$ that is 750,000.25. S2 needs at most 20. Per search, S2 is therefore about 37,500 times cheaper.

What it means. Each doubling of the catalogue adds one comparison to S2, so growing from 100,000 to 1,000,000 products costs only 3 more comparisons (17 to 20). S2 is the only version that scales. The recursive form uses $\Theta(\log n)$ stack space (about 20 frames) and the iterative form uses $\Theta(1)$, so the iterative form is slightly preferable. The price of S2 is keeping the index sorted when products are added.

4.4 M4: Recommendation Engine

What it does. It splits the customer-product data into parts and merges the partial results in linear time. The current design uses four parts of size n/2 because of replicated partitions. The proposed redesign uses two.

Results.

Design	Recurrence	Master Theorem	Complexity	Exact count
Current	$T(n) = 4T(n/2) + n$	$a = 4, b = 2, n^{(\log_2 4)} = n^2, f(n) = n$, Case 1	$\Theta(n^2)$	$2n^2 - n$
Redesign	$T(n) = 2T(n/2) + n$	$a = 2, b = 2, n^{(\log_2 2)} = n, f(n) = n$, Case 2	$\Theta(n \log n)$	$n \log n + n$

What happens when the data doubles. The current design becomes about 4 times slower. The redesign becomes only slightly more than 2 times slower (about 2.1 times at this size).

n	Current design (10^8 ops/s)	Redesign (10^8 ops/s)
1,000,000	20,000 s (5.6 hours)	0.21 s
2,000,000	80,000 s (22 hours)	0.44 s

What it means. The replication is the cause. Extra copies of the data multiply the number of subproblems, and that moves the recurrence from the n log n regime into the n^2 regime. Removing the replication makes the redesign about 95,500 times cheaper at one million products.

4.5 M5: Sales Analytics Engine and Report Generator

Analytics engine. It splits sales data in half recursively and sorts at every level: $T(n) = 2T(n/2) + n \log n$.

None of the three standard Master Theorem cases applies. The function n log n is bigger than n (so not Case 1 or 2) but not polynomially bigger (so not Case 3). The extended Master Theorem with k = 1, or a recursion tree, gives

$T(n) = (n \log^2 n + n \log n)/2 + n = \Theta(n \log^2 n)$

At one million records this is about 2.1×10^8 operations, or 2.1 seconds. It fits the window comfortably. Sorting at every level costs an extra factor of log n (about 20), but that is affordable here.

Report generator. Profiling shows $f(n) = 3n^2 + 10n \log n + 50$. This is $\Theta(n^2)$, proved formally with constants $c_1 = 3$, $c_2 = 14$ and $n_0 = 8$. It is not O(n), because f(n)/n grows without limit.

What it means. The quadratic term makes up over 99.99% of the cost at one million records (3×10^{12} of 3.0002×10^{12}). At 10^8 ops/s the report takes 8.3 hours. The remaining terms are negligible, so only the quadratic step is worth optimising.

5. The Combined Nightly Workload

The three heavy jobs share one nightly window. Taking them together (running one after another at 10^8 ops/s):

	M1	M4 current	Report generator	Total	Share of 600 s window
n = 100,000 (today)	50 s	200 s	300 s	550 s	92%
n = 1,000,000 (next year)	5,000 s	20,000 s	30,002 s	55,002 s (15.3 hours)	9,167%

The system works today only because it is small. Next year the same jobs would need over 91 times the entire window. If the servers were 10 times faster (10^9), the total would still be about 5,500 seconds, which is more than nine times the window.

6. How Soon Does Each Module Break?

Because the growth is quadratic, the failures arrive sooner than the one-million figure suggests. The table shows the catalogue size at which each module alone uses the full 10-minute window.

Module	Breaks the window at n ≈ (10^8 ops/s)	Growth from today	Breaks at n ≈ (10^9 ops/s)
Report generator	141,000	+41%	447,000
M4 current design	173,000	+73%	548,000
M1 duplicate detector	346,000	3.5 times	1,095,000
M5 analytics engine	about 156 million	far away	far away
M4 redesign	far above 10^8	far away	far away

Illustrative timing. The catalogue's growth pattern is not given, so these are rough guides only. They assume the 100,000 to 1,000,000 growth happens over 365 days, either steadily (linear) or compounding (exponential), at 10^8 ops/s.

Module	Steady growth: breaks after about	Compounding growth: breaks after about
Report generator	17 days	55 days
M4 current design	30 days	87 days
M1 duplicate detector	100 days	197 days

Either way, the report generator and the M4 design are the urgent ones. M1 has more time, but its growth is just as steep.

Projected nightly time as the catalogue grows (10^8 ops/s):

n	M1	M4 current	Report generator	M4 redesign	M5 analytics	M2
100,000	50 s	200 s	300 s	0.02 s	0.15 s	0.001 s
200,000	200 s	800 s	1,200 s	0.04 s	0.33 s	0.003 s
500,000	1,250 s	5,000 s	7,501 s	0.10 s	0.95 s	0.005 s
1,000,000	5,000 s	20,000 s	30,002 s	0.21 s	2.10 s	0.011 s
2,000,000	20,000 s	80,000 s	120,004 s	0.44 s	4.61 s	0.021 s

7. Sensitivity to Server Speed

The server speed has a blank exponent in the question paper, so I checked both likely values at n = 1,000,000.

Module	10^8 ops/s	Within 10 min?	10^9 ops/s	Within 10 min?
M1	5,000 s	No	500 s	Barely (8.3 min)

M4 current	20,000 s	No	2,000 s	No
Report generator	30,002 s	No	3,000 s	No
M4 redesign	0.21 s	Yes	0.02 s	Yes
M5 analytics	2.1 s	Yes	0.21 s	Yes
M2	0.01 s	Yes	0.001 s	Yes

Conclusion on sensitivity. M4 (current) and the report generator fail under both speeds. M1 fails at 10^8 and passes at 10^9 only by a small margin, and it still contributes to a total workload that far exceeds the window. Buying faster hardware does not solve the problem, because the cost grows with n^2, so a ten-fold faster server only absorbs a ten-fold growth in work, which is about three-fold growth in n. The conclusion (redesign these three modules) holds either way.

8. Recommendations

8.1 Redesign plan

Priority	Module	Change	New complexity	New cost at n = 1,000,000	Expected improvement
1	Report generator	Find the quadratic step (the source of the $3n^2$ term) and replace it with sorting, hashing or a single aggregation pass	$\Theta(n \log n)$	about 2×10^8 ops, about 2 s	about 15,000 times fewer operations
2	M4 recommendation	Adopt the proposed redesign that removes replicated partitions	$\Theta(n \log n)$	about 2.1×10^7 ops, about 0.2 s	about 95,500 times fewer operations

3	M1 duplicates	Hash each product by (title, seller) and flag a product when its key is already present	$\Theta(n)$ expected	about 10^6 ops, about 0.01 s	about 500,000 times fewer operations

M1 alternatives. If memory is tight, sort the products by (title, seller) and compare neighbours only. That is $\Theta(n \log n)$, about 2×10^7 operations, still about 25,000 times fewer than the current approach.

Report generator caveat. I do not know exactly what the report computes, so the estimate assumes its quadratic step can be replaced by an n log n method. If the report genuinely needs every pair of records, the target would need to be reconsidered, for example with pre-aggregation or incremental updates so the work is not redone every night.

8.2 Modules to keep

Module	Reason
M2	$\Theta(n)$, about 10^6 operations. The claim that it is $\Theta(n \log n)$ was wrong, so no redesign is needed.
M3 (S2)	About 20 comparisons per search. Use the iterative version to save stack space. Keep the index sorted when products are added.
M5 analytics	About 2.1 s. Optional tuning: sort once and merge by revenue at each level instead of re-sorting, which gives $T(n) = 2T(n/2) + n = \Theta(n \log n)$ and about 10 times fewer operations.

8.3 Expected result after redesign

Estimated nightly workload at n = 1,000,000 and 10^8 ops/s, with every recommendation applied:

Job	Operations	Time
M1 (hash method)	about 1.0×10^6	about 0.01 s
M2	about 1.05×10^6	about 0.01 s

M4 (redesign)	about 2.09×10^7	about 0.21 s
M5 analytics	about 2.10×10^8	about 2.10 s
Report generator (assuming $\Theta(n \log n)$)	about 2.0×10^8	about 2.0 s
Total	about 4.3×10^8	about 4.3 s

That is about 0.7% of the 10-minute window, compared with over 15 hours at present. The catalogue could grow far beyond one million products before this workload becomes a concern.

9. Risks and Trade-offs

Risk	Explanation	Mitigation
Extra memory in M1	The hash method needs $\Theta(n)$ memory, where the current routine needs $\Theta(1)$	One million small entries is modest for a modern server; if needed, use the sort-and-scan option
Hash collisions	In the worst case, a poor hash function would make lookups slow	Use a good hash on the combined (title, seller) key and a prime table size; fall back to sorting if needed
Loss of replication in M4	The replicated partitions may have been providing availability or fault tolerance	Provide that protection some cheaper way, such as replicas of the finished results instead of the working data
Report redesign unknown	The report's real logic is not visible from the cost figure alone	Profile first to find the exact quadratic step before committing to a design
Index upkeep for S2	The sorted index must stay in order as new products arrive	Insert in batches or merge new products nightly
Constant factors	Real operations differ in cost, so the times are estimates	Re-measure on the real system after each change

10. Verification

I checked the derivations by running the algorithms and formulas as code (Python_Code.py). Every check passed.

Check	Result
M1 loop count equals $n(n-1)/2$ ($n = 2, 5, 10, 50, 200$)	Passed
M2 loop count equals $2^{\lceil \log n \rceil} - 1$, and lies between $n - 1$ and $2n - 3$	Passed
M2 at $n = 1{,}024$	Count = 1,023, as derived (the wrong claim would give 10,240)
S2: $T(n) = \lfloor \log n \rfloor + 1$ for every n from 1 to 4,999	Passed
M4 current: $T(n) = 2n^2 - n$ ($n = 1$ to 4,096)	Passed
M4 redesign: $T(n) = n \log n + n$ ($n = 1$ to 4,096)	Passed
M5: $T(n) = n \log n (\log n + 1)/2 + n$ ($n = 1$ to 4,096)	Passed
$3n^2 \leq f(n) \leq 14n^2$ for every n from 8 to 19,999	Passed
$f(n)/n$ at $n = 10^3, 10^5, 10^7$	3,100; 300,166; 30,000,233 (grows without limit, so f is not O(n))
S1 average at $n = 1{,}000$, $p = 0.5$	Simulation 749.3, formula 750.25 (within 0.2%)

These checks confirm the maths. They do not measure the real ShopSphere system. The running times come from the operation counts and the assumed server speed.

11. Limitations

- Server speed. The exponent is missing in the question paper. I assumed 10^8 operations per second and showed 10^9 as well.
- One cost per operation. Real systems have memory, disk and network costs that vary. The figures show the *shape* of the growth reliably, but the exact times are estimates.

- Sequential jobs on one server. Running jobs in parallel or on several servers would change the combined-workload figures, though not the growth rates.
- Unknown growth pattern. The timing table in Section 6 assumes two simple growth shapes. The real growth may differ.
- Report generator internals. Its logic was not provided, so the redesign target is a reasoned estimate and not a tested result.
- No measurement on real data. Everything is analytical or code-checked. No production data was available.

12. Conclusion

ShopSphere's back end has a clear split. Modules with cost that grows linearly or logarithmically (M2, S2, M5 analytics, and the M4 redesign) will handle a ten-fold growth easily. Three modules with quadratic cost (M1, the current M4, and the report generator) will not. They are already using 92% of the nightly window at today's size and will be hundreds of times over it at one million products. The report generator is the most urgent, because it would hit the window at about 141,000 products.

The fix does not need new hardware. It needs better algorithms: hashing for the duplicate detector, the already-proposed redesign for the recommendation engine, and a $\Theta(n \log n)$ method for the report generator. Together these bring the whole nightly workload at one million products to about four seconds.

Appendix A: Summary of Results

Item	Result
M1 comparisons	$n(n-1)/2$, so $\Theta(n^2)$, space $\Theta(1)$
M2 Audit operations	$2^{\lceil \log n \rceil} - 1$, so $\Theta(n)$, space $\Theta(1)$; the claim $\Theta(n \log n)$ is incorrect
S2 recurrence solution	$T(n) = \lfloor \log n \rfloor + 1$, so $\Theta(\log n)$; iterative space $\Theta(1)$, recursive space $\Theta(\log n)$

M4 current	$\Theta(n^2)$, exact $2n^2 - n$ (Master Theorem Case 1)
M4 redesign	$\Theta(n \log n)$, exact $n \log n + n$ (Master Theorem Case 2)
M5 recurrence	$\Theta(n \log^2 n)$ (extended Master Theorem, $k = 1$)
Report generator	$f(n) = \Theta(n^2)$, with $c_1 = 3$, $c_2 = 14$, $n_0 = 8$; not $O(n)$
S1 average	$n(1 - p/2) + p/2$, which is 750,000.25 for $n = 10^6$, $p = 0.5$
Modules failing the 10-minute window	M1 (at 10^8), M4 current, report generator

Appendix B: Where Each Question Part Is Answered

Part	Topic	Location
(a)	Input size and basic operation	Solution.pdf (a); Report Section 4
(b)	M1 and M2 exact counts	Solution.pdf (b); Report Sections 4.1 and 4.2
(c)	S2 backward substitution	Solution.pdf (c); Report Section 4.3
(d)	Master Theorem for M4 and M5	Solution.pdf (d); Report Sections 4.4 and 4.5
(e)	Best, average and worst cases	Solution.pdf (e); Report Section 4.3
(f)	$\Theta(n^2)$ proof for $f(n)$	Solution.pdf (f); Report Section 4.5
(g)	Scalability decision	Solution.pdf (g); Report Sections 5 to 8